

Polynomials

Why are polynomials interesting?

A polynomial P:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

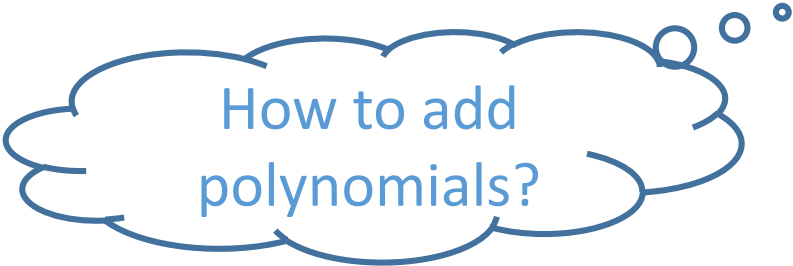
A polynomial Q:

$$Q(X) = q_0 + q_1 \cdot X^1 + q_2 \cdot X^2 + q_3 \cdot X^3 + \dots + q_{n-1} \cdot X^{n-1}$$

What are the common operations on polynomials ?

Addition,
Multiplication

Polynomials



How to add
polynomials?

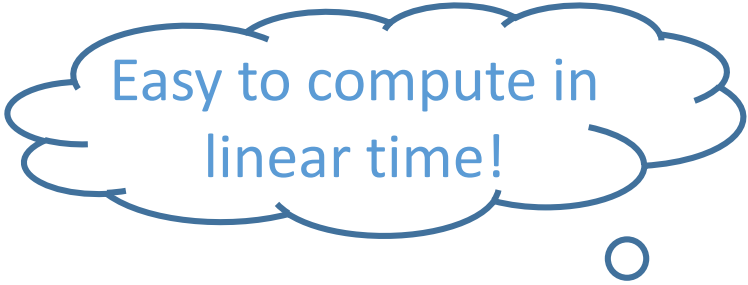
A polynomial P:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \cdots + p_{n-1} \cdot X^{n-1}$$

A polynomial Q:

$$Q(X) = q_0 + q_1 \cdot X^1 + q_2 \cdot X^2 + q_3 \cdot X^3 + \cdots + q_{n-1} \cdot X^{n-1}$$

$$\begin{aligned} &(P + Q)(X) \\ &= (p_0 + q_0) + (p_1 + q_1) \cdot X^1 + (p_2 + q_2) \cdot X^2 + \cdots + (p_{n-1} + q_{n-1}) \cdot X^{n-1} \end{aligned}$$



Easy to compute in
linear time!

Multiplication of Polynomials

How to multiply
polynomials?

A polynomial P:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

A polynomial Q:

$$Q(X) = q_0 + q_1 \cdot X^1 + q_2 \cdot X^2 + q_3 \cdot X^3 + \dots + q_{n-1} \cdot X^{n-1}$$

$$(P \times Q)(X)$$

$$= (p_0 \cdot q_0) + (p_0 \cdot q_1 + p_1 \cdot q_0) \cdot X^1 + (p_0 \cdot q_2 + p_1 \cdot q_1 + p_2 \cdot q_0) \cdot X^2 + \dots$$

$$+ (p_0 \cdot q_{\frac{n}{2}} + p_1 \cdot q_{\frac{n}{2}-1} + p_2 \cdot q_{\frac{n}{2}-2} + \dots + p_{\frac{n}{2}} \cdot q_0) \cdot X^{\frac{n}{2}} + \dots$$

$$+ (p_{n-1} \cdot q_{n-1}) \cdot X^{2n-2}$$

Takes $\Theta(n^2)$
time to compute!

Multiplication of Polynomials

How to multiply
polynomials?

A polynomial P:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

A polynomial Q:

$$Q(X) = q_0 + q_1 \cdot X^1 + q_2 \cdot X^2 + q_3 \cdot X^3 + \dots + q_{n-1} \cdot X^{n-1}$$

How can we multiply in lower
cost?

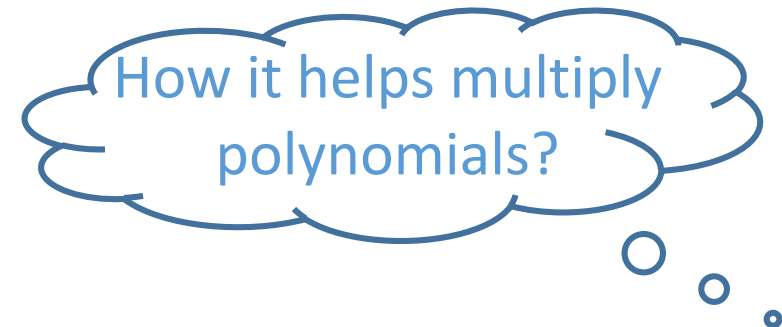
Make use of points
representation!

Points Representation of Polynomials

Theorem: (Algebra)

Given n distinct points $(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{C}$, there exists a single polynomial P of degree n that passes through the points $(x_1, y_1), \dots, (x_n, y_n)$.

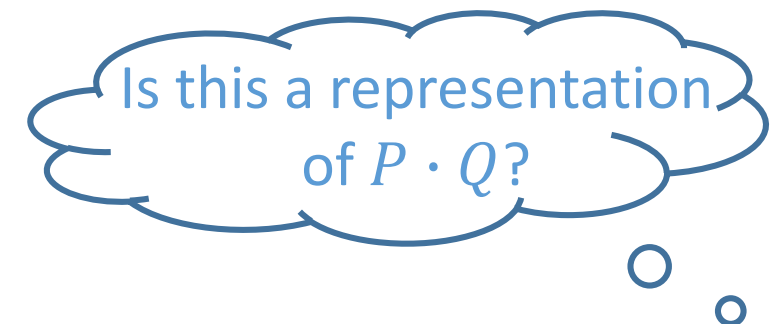
For example, only a single line passes through two given points



Multiplication of Polynomials

Example:

Polynomial	$x_1 = 0$	$x_2 = 1$		
$P(x) = 2x + 1$	1	3		
$Q(x) = x - 1$	-1	0		
$(P \cdot Q)(x)$ $= 2x^2 - x - 1$	-1	0		



Multiplication of Polynomials

Example:

Polynomial	$x_1 = 0$	$x_2 = 1$	$x_3 = 2$	$x_4 = 3$
$P(x) = 2x + 1$	1	3	5	7
$Q(x) = x - 1$	-1	0	1	2
$(P \cdot Q)(x)$ $= 2x^2 - x - 1$	-1	0	5	14

Takes $\Theta(n)$
time to multiply!

Multiplication of Polynomials

Example:

Polynomial	$x_1 = 0$	$x_2 = 1$	$x_3 = 2$	$x_4 = 3$
$P(x) = 2x + 1$	1	3	5	7
$Q(x) = x - 1$	-1	0	1	2
$(P \cdot Q)(x)$ $= 2x^2 - x - 1$	-1	0	5	14

But takes $\Theta(n)$ time to *efficiently* evaluate
each point, so in total $\Theta(n^2)$ time!

Takes $\Theta(n)$
time to multiply!

Multiplication of Polynomials

Example:

Polynomial	$x_1 = 0$	$x_2 = 1$	$x_3 = 2$	$x_4 = 3$
$P(x) = 2x + 1$	1	3	5	7
$Q(x) = x - 1$	-1	0	1	2
$(P \cdot Q)(x)$ $= 2x^2 - x - 1$	-1	0	5	14

FFT algorithm enables to *efficiently* evaluate
all points in total $\Theta(n \log n)$ time!

Takes $\Theta(n)$
time to multiply!

FFT – Fast Fourier Transform

Fourier Transform:

$$F^n: \mathbb{C}^n \rightarrow \mathbb{C}^n$$

Coefficients
representation of the
polynomial

Point representation
of the polynomial

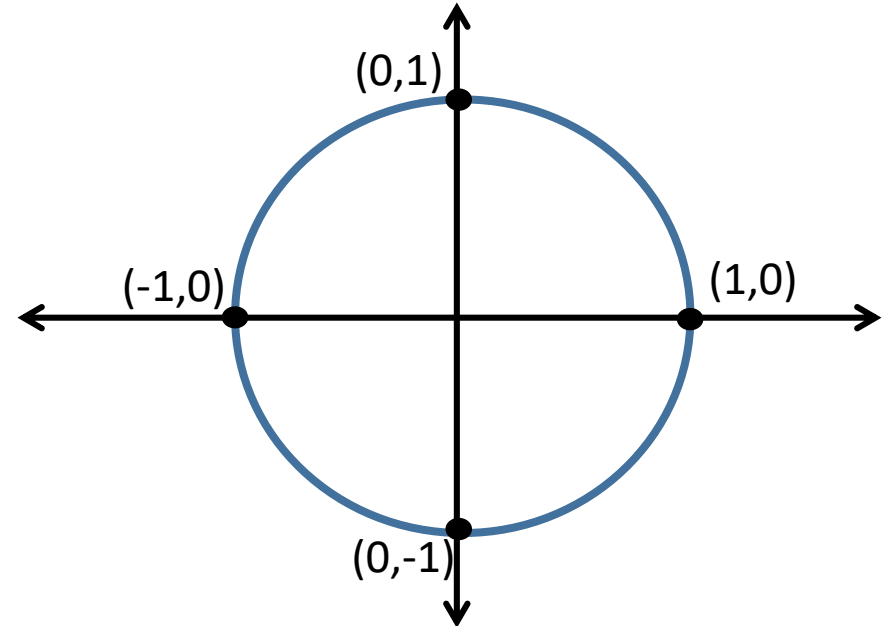
*In order to understand how FFT works we should have
basic knowledge about complex numbers!*

The Complex Field - Basics

Fact: (Algebra)

Every polynomial of degree n has exactly n complex roots.

We have special interest in the polynomial $P(x) = X^n - 1$.
The roots of this polynomial are called **unit roots**, and are the solutions of the equation: $X^n = 1$.



The Complex Field - Basics

Algebraic representation of complex numbers:

A point (x, y) in the complex plane is the complex number $x + i \cdot y$, where i is a complex number such that: $i^2 = -1$.

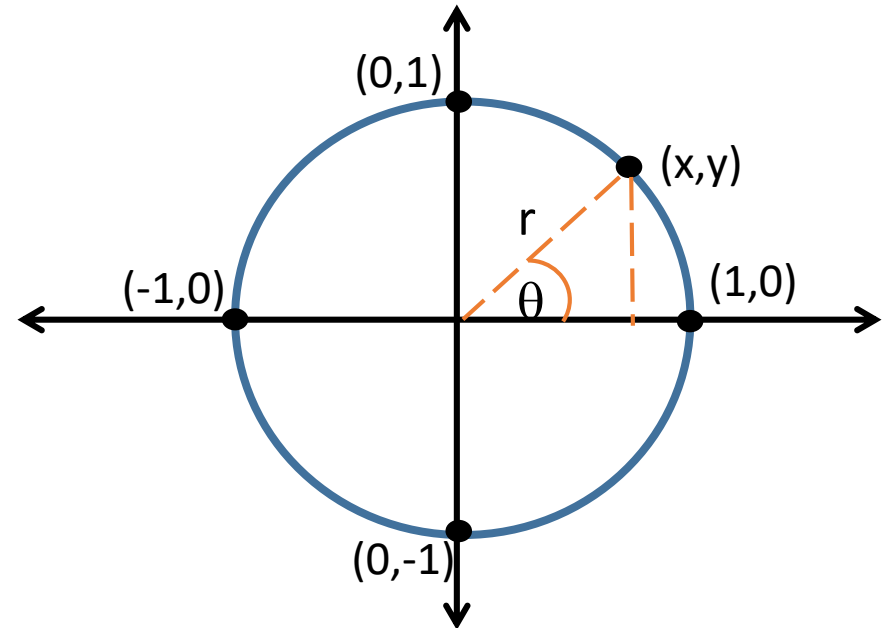
Polar representation of complex numbers: (r, θ) .

$$r = \sqrt{x^2 + y^2}$$

$$\theta = \tan^{-1} \frac{y}{x}$$

$$x = r \cdot \cos \theta$$

$$y = r \cdot \sin \theta$$



The Complex Field - Basics

Multiplication in polar representation:

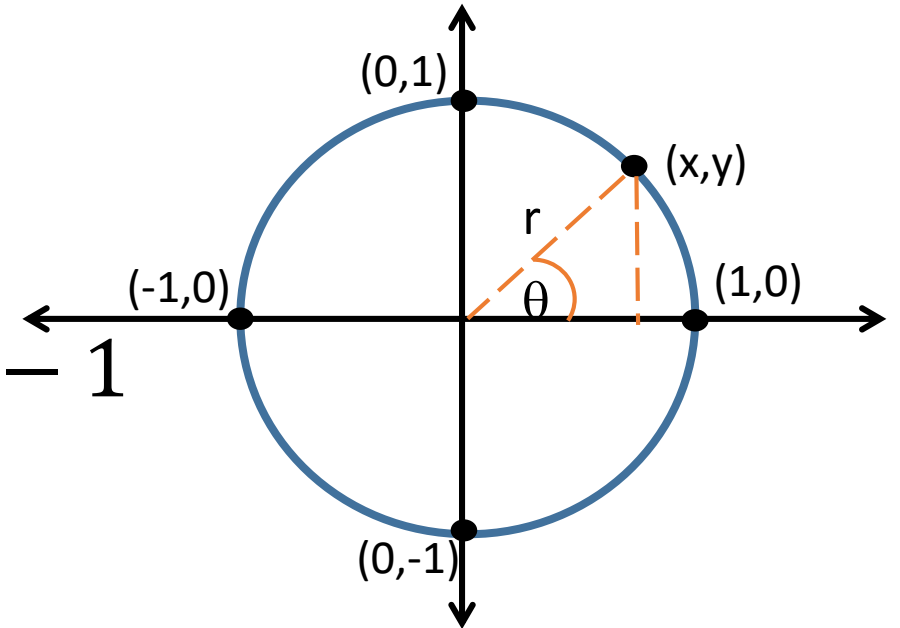
$$(r_1, \theta_1) \cdot (r_2, \theta_2) = (r_1 \cdot r_2, \theta_1 + \theta_2)$$

The unit roots are the solutions for the equation $X^n = 1$, so:

$$(r, \theta)^n = (r^n, n \cdot \theta) = (1, 0)$$

$$r^n = 1 \Rightarrow r = 1$$

$$n \cdot \theta = 0 = 2\pi k \Rightarrow \theta = \frac{2\pi k}{n}, k = 0, \dots, n-1$$



The Complex Field - Basics

Multiplication in polar representation:

$$(r_1, \theta_1) \cdot (r_2, \theta_2) = (r_1 \cdot r_2, \theta_1 + \theta_2)$$

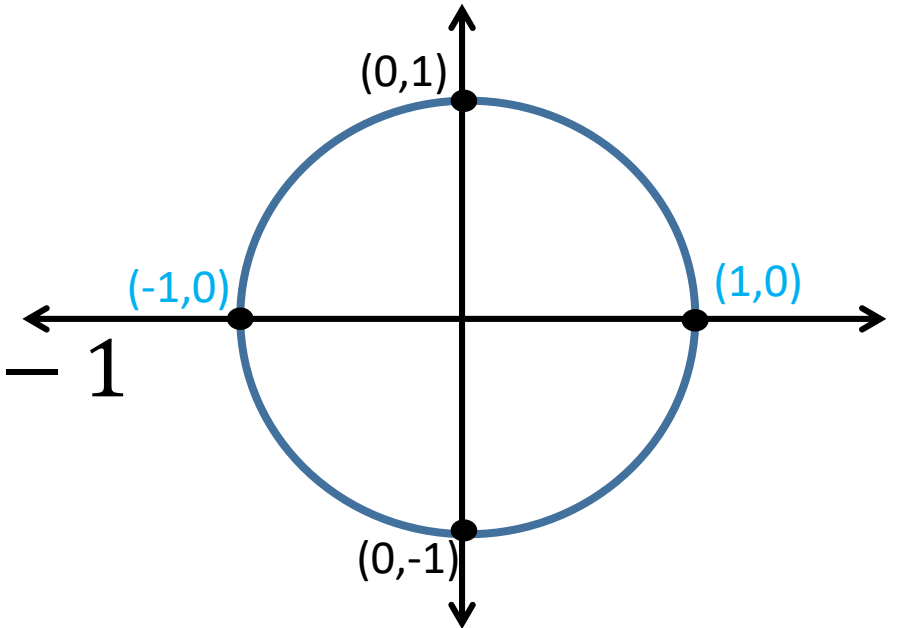
The unit roots are the solutions for the equation $X^n = 1$, so:

$$(r, \theta)^n = (r^n, n \cdot \theta) = (1, 0)$$

$$r^n = 1 \Rightarrow r = 1$$

$$n \cdot \theta = 0 = 2\pi k \Rightarrow \theta = \frac{2\pi k}{n}, k = 0, \dots, n-1$$

Example: unit roots for $n=2$ are $1, -1$



The Complex Field - Basics

Multiplication in polar representation:

$$(r_1, \theta_1) \cdot (r_2, \theta_2) = (r_1 \cdot r_2, \theta_1 + \theta_2)$$

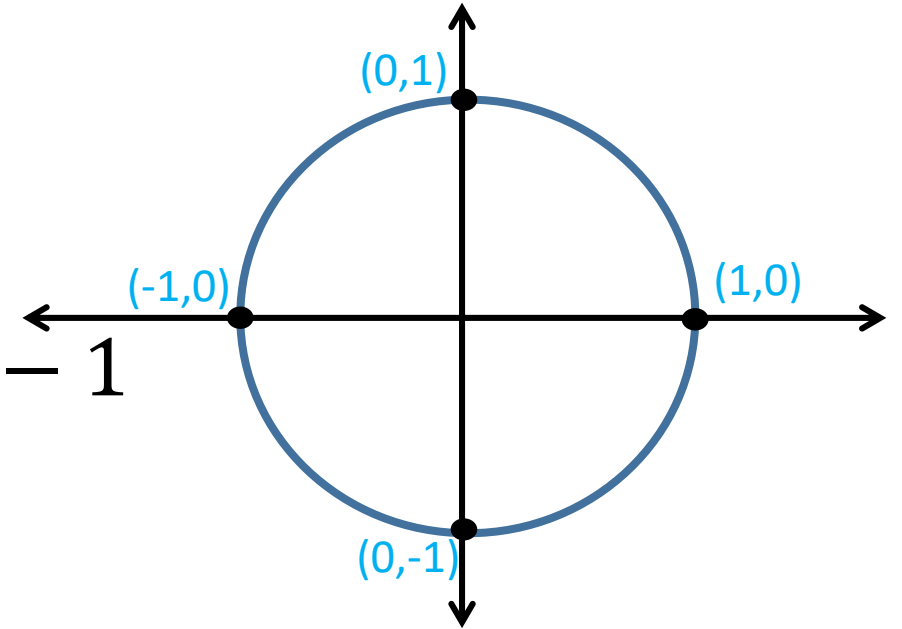
The unit roots are the solutions for the equation $X^n = 1$, so:

$$(r, \theta)^n = (r^n, n \cdot \theta) = (1, 0)$$

$$r^n = 1 \Rightarrow r = 1$$

$$n \cdot \theta = 0 = 2\pi k \Rightarrow \theta = \frac{2\pi k}{n}, k = 0, \dots, n-1$$

Example: unit roots for $n=4$ are $1, i, -1, -i$



The Complex Field - Basics

Multiplication in polar representation:

$$(r_1, \theta_1) \cdot (r_2, \theta_2) = (r_1 \cdot r_2, \theta_1 + \theta_2)$$

The unit roots are the solutions for the equation $X^n = 1$, so:

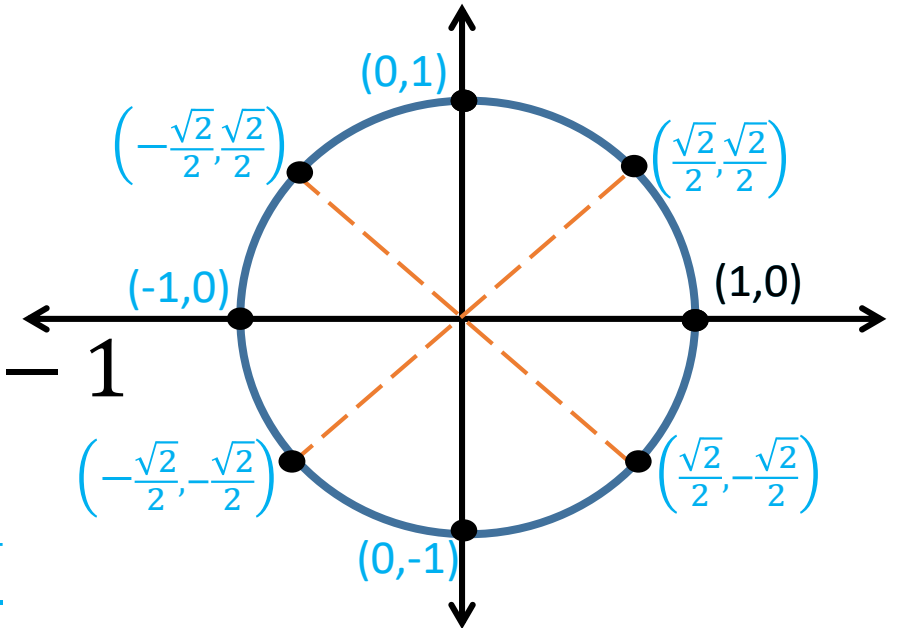
$$(r, \theta)^n = (r^n, n \cdot \theta) = (1, 0)$$

$$r^n = 1 \Rightarrow r = 1$$

$$n \cdot \theta = 0 = 2\pi k \Rightarrow \theta = \frac{2\pi k}{n}, k = 0, \dots, n-1$$

Example: unit roots for $n=8$ are

$$1, \frac{\sqrt{2}}{2} + i \cdot \frac{\sqrt{2}}{2}, i, -\frac{\sqrt{2}}{2} + i \cdot \frac{\sqrt{2}}{2}, -1, -\frac{\sqrt{2}}{2} - i \cdot \frac{\sqrt{2}}{2}, -i, \frac{\sqrt{2}}{2} - i \cdot \frac{\sqrt{2}}{2}$$



Properties of Unit Roots

Property 1:

If ω is a unit root of order n , then ω^j is a unit root of order n for every $j \in \mathbb{N}$.

Proof:

$$(\omega^j)^n = (\omega^n)^j = 1^j = 1$$

Primitive Unit Roots

Definition:

A unit root ω of order n is called a *primitive unit root*, if:
 $\omega^0, \omega^1, \omega^2, \dots, \omega^{n-1}$ are **different** numbers.

Example: unit roots for $n=2$ are $1, -1$

However, 1 is non-primitive while -1 is a primitive unit root, since

$$(-1)^0 = 1 \neq (-1)^1 = -1$$

Primitive Unit Roots

Definition:

A unit root ω of order n is called a *primitive unit root*, if:
 $\omega^0, \omega^1, \omega^2, \dots, \omega^{n-1}$ are **different** numbers.

Example: unit roots for $n=4$ are $1, i, -1, -i$

-1 is non-primitive (for order 4), since

$$(-1)^0 = 1 = (-1)^2$$

i is primitive (for order 4), since $i^0 = 1, i^1 = i, i^2 = -1, i^3 = -i$
are all different numbers.

Note that with the exponents of a primitive root of order n we get all unit root of order n !

Properties of Primitive Unit Roots

Property 2:

If ω is a *primitive unit root* of order n , let $c \in \mathbb{Z}, c \neq 0$, such that n does not divide c , then:

$$\sum_{k=0}^{n-1} (\omega^c)^k = 0.$$

Proof:

$\sum_{k=0}^{n-1} (\omega^c)^k$ is a geometric series. Thus,

$$\sum_{k=0}^{n-1} (\omega^c)^k = \frac{(\omega^c)^n - 1}{\omega^c - 1} = \frac{(\omega^n)^c - 1}{\omega^c - 1} = \frac{1^c - 1}{\omega^c - 1} = \frac{1 - 1}{\omega^c - 1} = 0$$

By geometric series
sum formula

By definition of
order n unit root

This is true only if
 $\omega^c - 1 \neq 0$

Properties of Primitive Unit Roots

Property 2:

If ω is a *primitive unit root* of order n , let $c \in \mathbb{Z}, c \neq 0$, such that n does not divide c , then:

$$\sum_{k=0}^{n-1} (\omega^c)^k = 0.$$

Proof:

$\sum_{k=0}^{n-1} (\omega^c)^k$ is a geometric series. Thus,

$$\sum_{k=0}^{n-1} (\omega^c)^k = \frac{(\omega^c)^n - 1}{\omega^c - 1} = \frac{(\omega^n)^c - 1}{\omega^c - 1} = \frac{1^c - 1}{\omega^c - 1} = \frac{1 - 1}{\omega^c - 1} = 0$$

By geometric series
sum formula

By definition of
order n unit root

This is true only if
 $\omega^c - 1 \neq 0$

$\omega^c - 1 \neq 0$ if and only if $\omega^c \neq 1$
But n does not divide c
and ω is a *primitive* root...

Properties of Primitive Unit Roots

Property 2:

If ω is a *primitive unit root* of order n , let $c \in \mathbb{Z}, c \neq 0$, such that n does not divide c , then:

$$\sum_{k=0}^{n-1} (\omega^c)^k = 0.$$

Proof:

$\sum_{k=0}^{n-1} (\omega^c)^k$ is a geometric series. Thus,

$$\sum_{k=0}^{n-1} (\omega^c)^k = \frac{(\omega^c)^n - 1}{\omega^c - 1} = \frac{(\omega^n)^c - 1}{\omega^c - 1} = \frac{1^c - 1}{\omega^c - 1} = \frac{1 - 1}{\omega^c - 1} = 0$$

By geometric series
sum formula

By definition of
order n unit root

This is true only if
 $\omega^c - 1 \neq 0$

Thus, if $0 < c < n$ then $\omega^c \neq 1$
since ω^c can only be one of:
 $\omega^1, \omega^2, \dots, \omega^{n-1} \neq \omega^0 = 1$

Properties of Primitive Unit Roots

Property 2:

If ω is a *primitive unit root* of order n , let $c \in \mathbb{Z}, c \neq 0$, such that n does not divide c , then: $\sum_{k=0}^{n-1} (\omega^c)^k = 0$.

Also, if $(j-1) \cdot n < c < j \cdot n$ then ω^c can only be one of:
 $\omega^{(j-1) \cdot n + 1}, \omega^{(j-1) \cdot n + 2}, \dots, \omega^{j \cdot n - 1} \neq \omega^0$
i.e., from $\omega^1, \omega^2, \dots, \omega^{n-1} \neq \omega^0 = 1$
since $\omega^{(j-1) \cdot n} = \omega^n = 1$

Proof:

$\sum_{k=0}^{n-1} (\omega^c)^k$ is a geometric series. Thus,

$$\sum_{k=0}^{n-1} (\omega^c)^k = \frac{(\omega^c)^n - 1}{\omega^c - 1} = \frac{(\omega^n)^c - 1}{\omega^c - 1} = \frac{1^c - 1}{\omega^c - 1} = \frac{1 - 1}{\omega^c - 1} = 0$$

By geometric series
sum formula

By definition of
order n unit root

This is true only if
 $\omega^c - 1 \neq 0$

Properties of Primitive Unit Roots

Property 2:

If ω is a *primitive unit root* of order n , let $c \in \mathbb{Z}, c \neq 0$, such that n does not divide c , then: $\sum_{k=0}^{n-1} (\omega^c)^k = 0$.

Note, that for negative $c \neq 0$ we have:

$-j \cdot n < c < -(j-1) \cdot n$ then

$\omega^c \neq \omega^0 = 1$ since as before:

$\omega^{-j \cdot n + 1}, \omega^{-j \cdot n + 2}, \dots, \omega^{-(j-1) \cdot n - 1} \neq \omega^0$

and $\omega^{-j \cdot n} = \omega^n = 1$

Proof:

$\sum_{k=0}^{n-1} (\omega^c)^k$ is a geometric series. Thus,

$$\sum_{k=0}^{n-1} (\omega^c)^k = \frac{(\omega^c)^n - 1}{\omega^c - 1} = \frac{(\omega^n)^c - 1}{\omega^c - 1} = \frac{1^c - 1}{\omega^c - 1} = \frac{1 - 1}{\omega^c - 1} = 0$$

By geometric series
sum formula

By definition of
order n unit root

This is true only if
 $\omega^c - 1 \neq 0$

Properties of Primitive Unit Roots

Conclusion:

The sum of all *unit root* of order $n \geq 2$, is 0.

Proof:

By property 2, by taking a primitive unit root of order $n \geq 2$, and $c = 1$, we have that n does not divide c , as required.

Note that the powers $k = 0, \dots, n - 1$ of a primitive root of order n generate all the unit root of order n .

Properties of Primitive Unit Roots

Property 3:

If ω is a *primitive unit root* of order n , where n is even, then:

ω^2 is a primitive unit root of order $\frac{n}{2}$.

Proof: $(\omega^2)^{\frac{n}{2}} = \omega^n = 1$

Therefore, ω^2 is a unit root of order $\frac{n}{2}$. We need to show that it is also primitive, i.e., $(\omega^2)^0, (\omega^2)^1, (\omega^2)^2, \dots, (\omega^2)^{\frac{n}{2}-1}$ are *different*.

Properties of Primitive Unit Roots

Property 3:

If ω is a *primitive unit root* of order n , where n is even, then:

ω^2 is a primitive unit root of order $\frac{n}{2}$.

Proof: $(\omega^2)^{\frac{n}{2}} = \omega^n = 1$

Therefore, ω^2 is a unit root of order $\frac{n}{2}$. We need to show that it is also primitive, i.e., $(\omega^2)^0, (\omega^2)^1, (\omega^2)^2, \dots, (\omega^2)^{\frac{n}{2}-1}$ are *different*.

Since ω is a *primitive unit root* of order n , we have that:

$\omega^0, \omega^1, \omega^2, \omega^3, \omega^4, \dots, \omega^{n-2}, \omega^{n-1}$ are *different*. In particular,

$(\omega^2)^0, (\omega^2)^1, (\omega^2)^2, \dots, (\omega^2)^{\frac{n}{2}-1}$ are *different*, as required.

Properties of Primitive Unit Roots

Property 4:

If ω is a *primitive unit root* of order n , where n is even, then:

$$\omega^{\frac{n}{2}} = -1.$$

Proof:

By property 3 we have that: $\left(\omega^{\frac{n}{2}}\right)^2 = (\omega^2)^{\frac{n}{2}} = 1$. Therefore, $\omega^{\frac{n}{2}}$ is a unit root of order 2. Thus, $\omega^{\frac{n}{2}} = 1$ or $\omega^{\frac{n}{2}} = -1$.

Properties of Primitive Unit Roots

Property 4:

If ω is a *primitive unit root* of order n , where n is even, then:

$$\omega^{\frac{n}{2}} = -1.$$

Proof:

By property 3 we have that: $\left(\omega^{\frac{n}{2}}\right)^2 = (\omega^2)^{\frac{n}{2}} = 1$. Therefore, $\omega^{\frac{n}{2}}$ is a unit root of order 2. Thus, $\omega^{\frac{n}{2}} = 1$ or $\omega^{\frac{n}{2}} = -1$.

However, ω is a *primitive unit root* of order n , so, in particular, $1 = \omega^0 \neq \omega^{\frac{n}{2}}$. Therefore, it must be that $\omega^{\frac{n}{2}} = -1$.

Properties of Primitive Unit Roots

Conclusion:

If ω is a *primitive unit root* of order n , where n is even, then the unit roots of order n are $\omega^0, \omega^1, \omega^2, \dots, \omega^{n-1}$, which are

$$\begin{array}{c} \omega^0, \omega^1, \omega^2, \dots, \omega^{\frac{n}{2}-1}, \omega^{\frac{n}{2}}, \omega^{\frac{n}{2}+1}, \dots, \omega^{n-1} \\ 1, \omega^1, \omega^2, \dots, \omega^{\frac{n}{2}-1}, -1, -\omega^1, \dots, -\omega^{\frac{n}{2}-1} \end{array}$$

Fourier Transform Definition

Fourier Matrix: F^n

The Fourier matrix of order n is built using a *primitive unit root* ω of order n as follows:

$$\begin{bmatrix} (\omega^0)^0 & \dots & (\omega^0)^{n-1} \\ \vdots & \ddots & \vdots \\ (\omega^{n-1})^0 & \dots & (\omega^{n-1})^{n-1} \end{bmatrix}$$

Who is the element in the i -th
row and j -th column?

$$(\omega^i)^j$$

Fourier Transform Definition

Fourier Matrix Example: F^4

The Fourier matrix of order 4 is built using a *primitive unit root* i of order 4 as follows:

$$\begin{bmatrix} (i^0)^0 & (i^0)^1 & (i^0)^2 & (i^0)^3 \\ (i^1)^0 & (i^1)^1 & (i^1)^2 & (i^1)^3 \\ (i^2)^0 & (i^2)^1 & (i^2)^2 & (i^2)^3 \\ (i^3)^0 & (i^3)^1 & (i^3)^2 & (i^3)^3 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix}$$

Who is the element in the i -th
row and j -th column?

$$(\omega^i)^j$$

Fourier Transform Definition

Fourier Transform: $F^n: \mathbb{C}^n \rightarrow \mathbb{C}^n$

Given a complex vector P of length n , the Fourier transform is the result of the multiplication of the Fourier matrix of order n by the vector P .

$$\begin{bmatrix} (\omega^0)^0 & \dots & (\omega^0)^{n-1} \\ \vdots & \ddots & \vdots \\ (\omega^{n-1})^0 & \dots & (\omega^{n-1})^{n-1} \end{bmatrix} \cdot \begin{bmatrix} p_0 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} p_0 + \dots + p_{n-1} \\ \vdots \\ p_0 + \dots + p_{n-1} \cdot (\omega^{n-1})^{n-1} \end{bmatrix} = \begin{bmatrix} P(\omega^0) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

The transform computes the values of the polynomial P on the *different* points

$\omega^0, \omega^1, \omega^2, \dots, \omega^{n-1}$

Fourier Transform Definition

Fourier Transform Example: $F^4: \mathbb{C}^4 \rightarrow \mathbb{C}^4$

Given a complex vector P of length 4, the Fourier transform is the result of the multiplication of the Fourier matrix of order 4 by the vector $P = (1, -2, 1, 3) = 1 - 2X + X^2 + 3X^3$.

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{bmatrix} \cdot \begin{bmatrix} 1 \\ -2 \\ 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 - 2 + 1 + 3 \\ 1 - 2i - 1 - 3i \\ 1 + 2 + 1 - 3 \\ 1 + 2i - 1 + 3i \end{bmatrix} = \begin{bmatrix} 3 \\ -5i \\ 1 \\ 5i \end{bmatrix} = \begin{bmatrix} P(1) \\ P(i) \\ P(-1) \\ P(-i) \end{bmatrix}$$

The transform computes the values of the polynomial P on the *different* points

$1, i, -1, -i$

Fourier Transform Definition

Fourier Transform: $F^n: \mathbb{C}^n \rightarrow \mathbb{C}^n$

Given a complex vector P of length n , the Fourier transform is the result of the multiplication of the Fourier matrix of order n by the vector P .

$$\begin{bmatrix} (\omega^0)^0 & \dots & (\omega^0)^{n-1} \\ \vdots & \ddots & \vdots \\ (\omega^{n-1})^0 & \dots & (\omega^{n-1})^{n-1} \end{bmatrix} \cdot \begin{bmatrix} p_0 \\ \vdots \\ p_{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} p_0 + \dots + p_{n-1} \\ \vdots \\ p_0 + \dots + p_{n-1} \cdot (\omega^{n-1})^{n-1} \end{bmatrix} = \begin{bmatrix} P(\omega^0) \\ \vdots \\ P(\omega^{n-1}) \end{bmatrix}$$

It transforms polynomial **P** from **coefficients** representation to **points** representation!

Takes $\Theta(n^2)$ time to multiply!

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \cdots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \cdots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \cdots + p_{n-1} \cdot X^{n/2-1}$$

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \cdots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \cdots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \cdots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \dots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \dots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

X should be $(\omega_n)^i$
for every $0 \leq i \leq n-1$

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \dots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \dots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

By recursion we have:

$$X = \left(\omega_{\frac{n}{2}}\right)^i \text{ for every } 0 \leq i \leq \frac{n}{2}-1$$

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \dots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \dots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

Recursion works due
to **Property 3**

ω_n^2 is a primitive unit
root of order $\frac{n}{2}$

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \dots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \dots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

Recursion works due
to **Property 3**

By recursion we have:

$$X = \left(\omega_{\frac{n}{2}}\right)^i \text{ for every } 0 \leq i \leq \frac{n}{2}-1$$

ω_n^2 is a primitive unit
root of order $\frac{n}{2}$

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \dots + p_{n-1} \cdot X^{n-1}$$

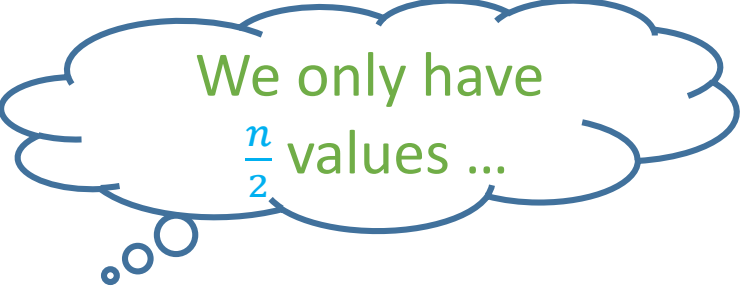
Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \dots + p_{n-2} \cdot X^{n/2-1}$$

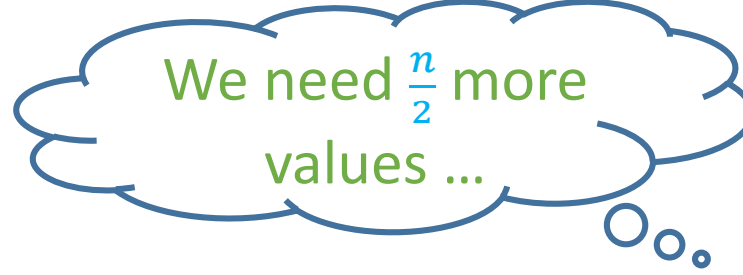
$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \dots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$



We only have
 $\frac{n}{2}$ values ...



We need $\frac{n}{2}$ more
values ...

FFT - Divide and Conquer

Assume n is a power of 2:

$$P(X) = p_0 + p_1 \cdot X^1 + p_2 \cdot X^2 + p_3 \cdot X^3 + \cdots + p_{n-1} \cdot X^{n-1}$$

Divide:

$$P_{\text{even}}(X) = p_0 + p_2 \cdot X^1 + p_4 \cdot X^2 + \cdots + p_{n-2} \cdot X^{n/2-1}$$

$$P_{\text{odd}}(X) = p_1 + p_3 \cdot X^1 + p_5 \cdot X^2 + \cdots + p_{n-1} \cdot X^{n/2-1}$$

Conquer:

$$P(X) = P_{\text{even}}(X^2) + X \cdot P_{\text{odd}}(X^2)$$

$$P(-X) = P_{\text{even}}(X^2) - X \cdot P_{\text{odd}}(X^2)$$

Missing values added
due to **Property 4**

Missing unit roots are
negatives of existing

FFT Algorithm

Remark:

Line 4 of the FFT pseudo-code computes:

$$e^{\frac{2\pi i}{n}}$$

This is actually the complex number $\cos \frac{2\pi}{n} + i \cdot \sin \frac{2\pi}{n}$.

Note, this computation returns a primitive unit root of order n .

Example: A primitive root for $n=2$ is -1 , we have that:

$$\cos \frac{2\pi}{2} + i \cdot \sin \frac{2\pi}{2} = -1 + i \cdot 0 = -1$$

A primitive root for $n=4$ is i , we have that:

$$\cos \frac{2\pi}{4} + i \cdot \sin \frac{2\pi}{4} = 0 + i \cdot 1 = i$$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow FFT(0,2)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow FFT(0,2)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(0,2)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0)$
- 7) $a^{[1]} \leftarrow (2)$
- 8) $y^{[0]} \leftarrow FFT(0)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow FFT(0,2)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(0,2)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0)$
- 7) $a^{[1]} \leftarrow (2)$
- 8) $y^{[0]} \leftarrow FFT(0)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(0)

- 1) $n \leftarrow 1$
- 2) if $n = 1$
- 3) return $a=0$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow FFT(0,2)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(0,2)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0)$
- 7) $a^{[1]} \leftarrow (2)$
- 8) $y^{[0]} \leftarrow (0)$
- 9) $y^{[1]} \leftarrow FFT(2)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow FFT(0,2)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(0,2)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0)$
- 7) $a^{[1]} \leftarrow (2)$
- 8) $y^{[0]} \leftarrow (0)$
- 9) $y^{[1]} \leftarrow FFT(2)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(2)

- 1) $n \leftarrow 1$
- 2) if $n = 1$
- 3) return $a=2$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow FFT(0,2)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(0,2)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0)$
- 7) $a^{[1]} \leftarrow (2)$
- 8) $y^{[0]} \leftarrow (0)$
- 9) $y^{[1]} \leftarrow (2)$
- 10) $\underline{k = 0}$
- 11) $y_0 \leftarrow 0 + 1 \cdot 2 = 2$
- 12) $y_1 \leftarrow 0 - 1 \cdot 2 = -2$
- 13) $\omega \leftarrow -1$
- 14) return $y = (2, -2)$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

1) $n \leftarrow 4$

4) $\omega_4 \leftarrow i$

5) $\omega \leftarrow 1$

6) $a^{[0]} \leftarrow (0,2)$

7) $a^{[1]} \leftarrow (1,3)$

8) $y^{[0]} \leftarrow (2, -2)$

9) $y^{[1]} \leftarrow FFT(1,3)$

10)

11)

12)

13)

14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow (2, -2)$
- 9) $y^{[1]} \leftarrow FFT(1,3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(1,3)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (1)$
- 7) $a^{[1]} \leftarrow (3)$
- 8) $y^{[0]} \leftarrow FFT(1)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow (2, -2)$
- 9) $y^{[1]} \leftarrow FFT(1,3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(1,3)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (1)$
- 7) $a^{[1]} \leftarrow (3)$
- 8) $y^{[0]} \leftarrow FFT(1)$
- 9)
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(1)

- 1) $n \leftarrow 1$
- 2) if $n = 1$
- 3) return $a=1$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow (2, -2)$
- 9) $y^{[1]} \leftarrow FFT(1,3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(1,3)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (1)$
- 7) $a^{[1]} \leftarrow (3)$
- 8) $y^{[0]} \leftarrow (1)$
- 9) $y^{[0]} \leftarrow FFT(3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow (2, -2)$
- 9) $y^{[1]} \leftarrow FFT(1,3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(1,3)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (1)$
- 7) $a^{[1]} \leftarrow (3)$
- 8) $y^{[0]} \leftarrow (1)$
- 9) $y^{[0]} \leftarrow FFT(3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(3)

- 1) $n \leftarrow 1$
- 2) if $n = 1$
- 3) return $a=3$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

- 1) $n \leftarrow 4$
- 4) $\omega_4 \leftarrow i$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (0,2)$
- 7) $a^{[1]} \leftarrow (1,3)$
- 8) $y^{[0]} \leftarrow (2, -2)$
- 9) $y^{[1]} \leftarrow FFT(1,3)$
- 10)
- 11)
- 12)
- 13)
- 14)

FFT(1,3)

- 1) $n \leftarrow 2$
- 4) $\omega_2 \leftarrow -1$
- 5) $\omega \leftarrow 1$
- 6) $a^{[0]} \leftarrow (1)$
- 7) $a^{[1]} \leftarrow (3)$
- 8) $y^{[0]} \leftarrow (1)$
- 9) $y^{[1]} \leftarrow (3)$
- 10) $\underline{k = 0}$
- 11) $y_0 \leftarrow 1 + 1 \cdot 3 = 4$
- 12) $y_1 \leftarrow 1 - 1 \cdot 3 = -2$
- 13) $\omega \leftarrow -1$
- 14) return $y = (4, -2)$

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

1) $n \leftarrow 4$

4) $\omega_4 \leftarrow i$

5) $\omega \leftarrow 1$

6) $a^{[0]} \leftarrow (0,2)$

7) $a^{[1]} \leftarrow (1,3)$

8) $y^{[0]} \leftarrow (2, -2)$

9) $y^{[1]} \leftarrow (4, -2)$

10) $k = 0$

11) $y_0 \leftarrow 2 + 1 \cdot 4 = 6$

12) $y_2 \leftarrow 2 - 1 \cdot 4 = -2$

13) $\omega \leftarrow i$

14)

FFT Algorithm

Example: $P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$

FFT(0,1,2,3)

1) $n \leftarrow 4$

4) $\omega_4 \leftarrow i$

5) $\omega \leftarrow 1$

6) $a^{[0]} \leftarrow (0,2)$

7) $a^{[1]} \leftarrow (1,3)$

8) $y^{[0]} \leftarrow (2, -2)$

9) $y^{[1]} \leftarrow (4, -2)$

10) $\underline{k = 0}$

11) $y_0 \leftarrow 2 + 1 \cdot 4 = 6$

12) $y_2 \leftarrow 2 - 1 \cdot 4 = -2$

13) $\omega \leftarrow i$

14) return $y = (6, -2 - 2i, -2, -2 + 2i)$

10) $\underline{k = 1}$

11) $y_1 \leftarrow -2 + i \cdot (-2) = -2 - 2i$

12) $y_3 \leftarrow -2 - i \cdot (-2) = -2 + 2i$

13) $\omega \leftarrow -1$

FFT Algorithm

Example:

$$P(X) = 1 \cdot X^1 + 2 \cdot X^2 + 3 \cdot X^3$$

FFT(0,1,2,3)

1) $n \leftarrow 4$

4) $\omega_4 \leftarrow i$

5) $\omega \leftarrow 1$

6) $a^{[0]} \leftarrow (0,2)$

7) $a^{[1]} \leftarrow (1,3)$

8) $y^{[0]} \leftarrow (2, -2)$

9) $y^{[1]} \leftarrow (4, -2)$

10) $\underline{k = 0}$

11) $y_0 \leftarrow 2 + 1 \cdot 4 = 6$

12) $y_2 \leftarrow 2 - 1 \cdot 4 = -2$

13) $\omega \leftarrow i$

14) return $y = (6, -2 - 2i, -2, -2 + 2i)$

Indeed, we have that:

$$P(1) = 1 \cdot 1^1 + 2 \cdot 1^2 + 3 \cdot 1^3 = 6$$

$$P(i) = 1 \cdot i^1 + 2 \cdot i^2 + 3 \cdot i^3 = -2 - 2i$$

$$P(-1) = 1 \cdot (-1)^1 + 2 \cdot (-1)^2 + 3 \cdot (-1)^3 = -2$$

$$P(-i) = 1 \cdot (-i)^1 + 2 \cdot (-i)^2 + 3 \cdot (-i)^3 = -2 + 2i$$

10) $\underline{k = 1}$

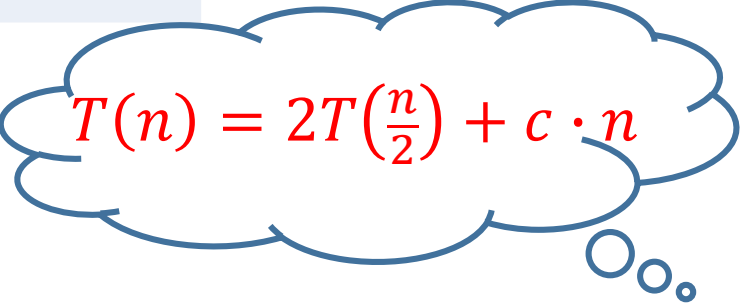
11) $y_1 \leftarrow -2 + i \cdot (-2) = -2 - 2i$

12) $y_3 \leftarrow -2 - i \cdot (-2) = -2 + 2i$

13) $\omega \leftarrow -1$

FFT Algorithm Cost

Command Number	Cost
1	$\Theta(n)$
2-5	$\Theta(1)$
6-7	$\Theta(n)$
8-9	$2T(\frac{n}{2})$
10-13	$\Theta(n)$
Total	$\Theta(n \log n)$


$$T(n) = 2T\left(\frac{n}{2}\right) + c \cdot n$$

FFT Algorithm Cost

$$T(n) = 2T\left(\frac{n}{2}\right) + c \cdot n$$

$$T(n) = 2 \left[2T\left(\frac{n}{2^2}\right) + c \cdot \frac{n}{2} \right] + c \cdot n = 2^2 T\left(\frac{n}{2^2}\right) + 2c \cdot n = \dots$$

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + ic \cdot n$$

$$T(n) = 2^{\log n} T\left(\frac{n}{2^{\log n}}\right) + \log n c \cdot n$$

$$T(n) = nT(1) + c \cdot n \log n = \Theta(n \log n)$$

Recursion stops when:

$$\frac{n}{2^i} = 1 \Leftrightarrow i = \log n$$

Back to Polynomials Multiplication...

Fast Polynomial Multiplication Scheme:

Input: polynomials P, Q of degree n .

- 1) Evaluate P, Q in $2n$ unit roots of order $2n$ by $\text{FFT}(P')$, $\text{FFT}(Q')$, where P', Q' are the coefficients vectors of P, Q padded with n leading zeroes.
- 2) Compute the point representation of the multiplication polynomial R by $R(\omega^i) = P'(\omega^i) \cdot Q'(\omega^i)$, for every $0 \leq i \leq 2n - 1$.
- 3) Compute the coefficient representation of R .



How to compute
step 3?

Back to Polynomials Multiplication...

Let V_R be the N -length vector of the point representation of R computed at step 2, where $N = 2n$. We know that:

$$F^N \cdot \overset{\text{unknown}}{R} = \overset{\text{known}}{V_R}$$



How to compute
step 3?

Back to Polynomials Multiplication...

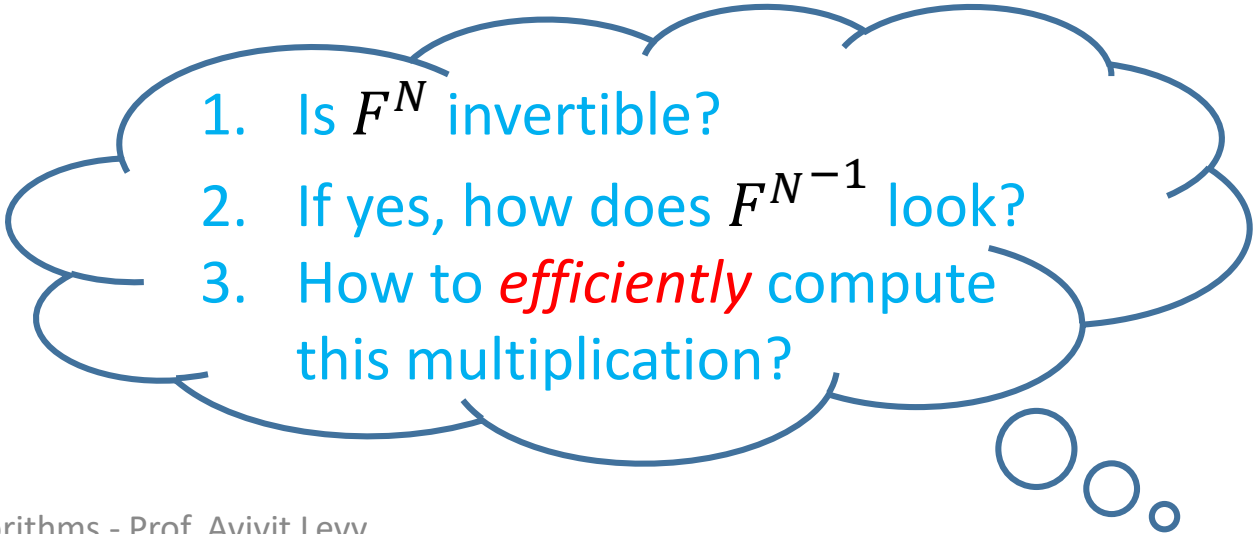
Let V_R be the N -length vector of the point representation of R computed at step 2, where $N = 2n$. We know that:

$$\begin{aligned} & \text{unknown } F^N \cdot R = \text{known } V_R \\ F^{N-1} \cdot F^N \cdot R &= F^{N-1} \cdot V_R \\ R &= F^{N-1} \cdot V_R \end{aligned}$$

Back to Polynomials Multiplication...

Let V_R be the N -length vector of the point representation of R computed at step 2, where $N = 2n$. We know that:

$$\begin{aligned} & \text{unknown } F^N \cdot \text{known } R = V_R \\ F^{N^{-1}} \cdot F^N \cdot R &= F^{N^{-1}} \cdot V_R \\ R &= F^{N^{-1}} \cdot V_R \end{aligned}$$

- 
1. Is F^N invertible?
 2. If yes, how does $F^{N^{-1}}$ look?
 3. How to *efficiently* compute this multiplication?

Back to Polynomials Multiplication...

Claim:

Let ω be a primitive unit root of order N . Then,

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Proof:

We need to show that $F^{N^{-1}} \cdot F^N = F^N \cdot F^{N^{-1}} = I$.

$$F^{N^{-1}} \cdot F^N[i, j] = \sum_{k=0}^{N-1} F^{N^{-1}}[i, k] \cdot F^N[k, j] =$$

Back to Polynomials Multiplication...

Claim:

Let ω be a primitive unit root of order N . Then,

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Proof:

We need to show that $F^{N^{-1}} \cdot F^N = F^N \cdot F^{N^{-1}} = I$.

$$\begin{aligned} F^{N^{-1}} \cdot F^N[i, j] &= \sum_{k=0}^{N-1} F^{N^{-1}}[i, k] \cdot F^N[k, j] = \\ &= \sum_{k=0}^{N-1} \frac{1}{N} \cdot \omega^{-ik} \cdot \omega^{kj} = \end{aligned}$$

Back to Polynomials Multiplication...

Claim:

Let ω be a primitive unit root of order N . Then,

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Proof:

We need to show that $F^{N^{-1}} \cdot F^N = F^N \cdot F^{N^{-1}} = I$.

$$\begin{aligned} F^{N^{-1}} \cdot F^N[i, j] &= \sum_{k=0}^{N-1} F^{N^{-1}}[i, k] \cdot F^N[k, j] = \\ &= \sum_{k=0}^{N-1} \frac{1}{N} \cdot \omega^{-ik} \cdot \omega^{kj} = \\ &= \frac{1}{N} \cdot \sum_{k=0}^{N-1} \omega^{k(j-i)} = \end{aligned}$$

Back to Polynomials Multiplication...

Claim:

Let ω be a primitive unit root of order N . Then,

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Proof:

We need to show that $F^{N^{-1}} \cdot F^N = F^N \cdot F^{N^{-1}} = I$.

$$\begin{aligned} F^{N^{-1}} \cdot F^N[i, j] &= \sum_{k=0}^{N-1} F^{N^{-1}}[i, k] \cdot F^N[k, j] = \\ &= \sum_{k=0}^{N-1} \frac{1}{N} \cdot \omega^{-ik} \cdot \omega^{kj} = \\ &= \frac{1}{N} \cdot \sum_{k=0}^{N-1} \omega^{k(j-i)} = \\ &= \frac{1}{N} \cdot \sum_{k=0}^{N-1} (\omega^{(j-i)})^k \end{aligned}$$

Back to Polynomials Multiplication...

Claim:

Let ω be a primitive unit root of order N . Then,

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Proof:

We need to show that $F^{N^{-1}} \cdot F^N = F^N \cdot F^{N^{-1}} = I$.

$$\begin{aligned} F^{N^{-1}} \cdot F^N[i, j] &= \sum_{k=0}^{N-1} F^{N^{-1}}[i, k] \cdot F^N[k, j] = \\ &= \sum_{k=0}^{N-1} \frac{1}{N} \cdot \omega^{-ik} \cdot \omega^{kj} = \\ &= \frac{1}{N} \cdot \sum_{k=0}^{N-1} \omega^{k(j-i)} = \\ &= \frac{1}{N} \cdot \sum_{k=0}^{N-1} (\omega^{(j-i)})^k = \begin{cases} 1, & \text{if } j = i \\ 0, & \text{if } j \neq i \end{cases} \end{aligned}$$

We got the
identity matrix

By
property 2

Back to Polynomials Multiplication...

Claim:

Let ω be a primitive unit root of order N . Then,

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Proof: (cont.)

Similarly we can show that $F^N \cdot F^{N^{-1}} = I$.

1. Is F^N invertible? YES!

2. If yes, how does $F^{N^{-1}}$ look? We know...

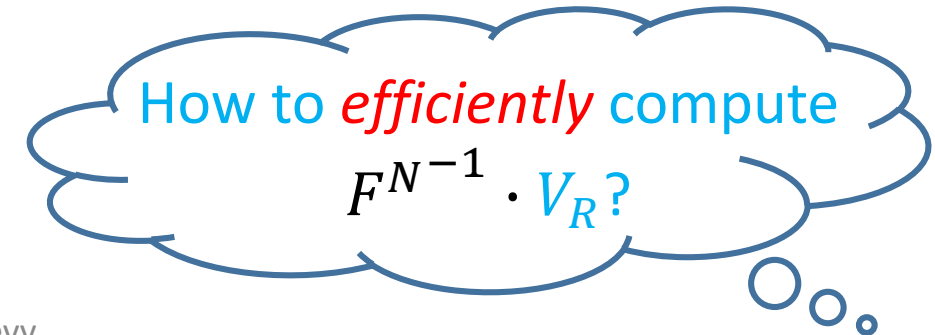
3. How to *efficiently* compute $F^{N^{-1}} \cdot V_R$?

Back to Polynomials Multiplication...

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Thus,

$$N \cdot F^{N^{-1}}[i, j] = \omega^{-ij}$$

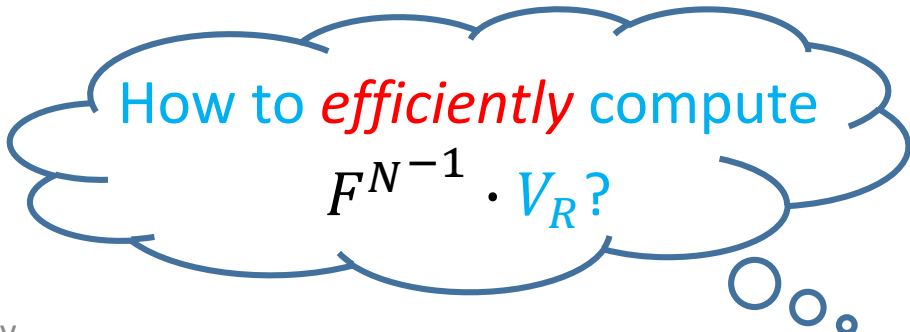


Back to Polynomials Multiplication...

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Thus,

$$N \cdot F^{N^{-1}}[i, j] = \omega^{-ij} = \frac{1}{\omega^{ij}} =$$



How to *efficiently* compute
 $F^{N^{-1}} \cdot V_R$?

Back to Polynomials Multiplication...

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Thus,

$$N \cdot F^{N^{-1}}[i, j] = \omega^{-ij} = \frac{1}{\omega^{ij}} = \frac{1^j}{\omega^{ij}} = \frac{(\omega^N)^j}{\omega^{ij}} =$$

How to *efficiently* compute

$$F^{N^{-1}} \cdot V_R?$$

Back to Polynomials Multiplication...

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Thus,

$$N \cdot F^{N^{-1}}[i, j] = \omega^{-ij} = \frac{1}{\omega^{ij}} = \frac{1^j}{\omega^{ij}} = \frac{(\omega^N)^j}{\omega^{ij}} = \frac{\omega^{Nj}}{\omega^{ij}}$$

How to *efficiently* compute

$F^{N^{-1}} \cdot V_R?$

Back to Polynomials Multiplication...

$$F^{N^{-1}}[i, j] = \frac{1}{N} \cdot \omega^{-ij}$$

Thus,

$$N \cdot F^{N^{-1}}[i, j] = \omega^{-ij} = \frac{1}{\omega^{ij}} = \frac{1^j}{\omega^{ij}} = \frac{(\omega^N)^j}{\omega^{ij}} = \frac{\omega^{Nj}}{\omega^{ij}} = \omega^{(N-i)j} = F^N[N - i, j]$$

Conclusion:

The i -th row of $F^{N^{-1}}$, $i > 0$, is the $(N-i)$ -th row of F^N divided by N .

The first row of $F^{N^{-1}}$ is the same as the first row of F^N divided by N .

How to *efficiently* compute

$$F^{N^{-1}} \cdot V_R?$$

Back to Polynomials Multiplication...

Inverse Fourier Matrix Example: $F^{4^{-1}}$

The inverse of Fourier matrix of order 4 is related to Fourier matrix of order 4 as follows:

$$F^4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -i \\ 1 & -i & -1 & i \end{bmatrix} \begin{matrix} \times \frac{1}{4} \\ \times \frac{1}{4} \\ \times \frac{1}{4} \\ \times \frac{1}{4} \end{matrix}$$
$$F^{4^{-1}} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & -\frac{i}{4} & -\frac{1}{4} & \frac{i}{4} \\ \frac{1}{4} & -\frac{1}{4} & \frac{1}{4} & -\frac{i}{4} \\ \frac{1}{4} & \frac{i}{4} & -\frac{1}{4} & -\frac{i}{4} \end{bmatrix}$$

Back to Polynomials Multiplication...

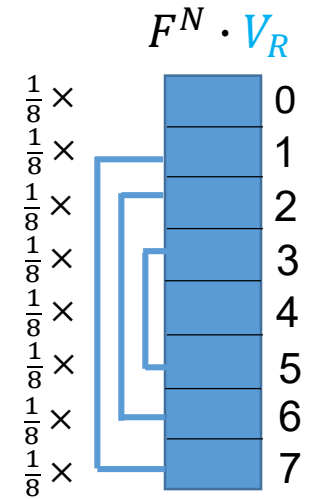
Conclusion:

In order to *efficiently* compute $F^{N^{-1}} \cdot V_R$:

$\Theta(N \log N)$ 1) Compute $F^N \cdot V_R$ using FFT

$\Theta(N)$ 2) In the result vector, exchange rows i and $N - i$ for $i = 1, \dots, \frac{N}{2} - 1$

$\frac{\Theta(N)}{\Theta(N \log N)}$ 3) Multiply each value by $\frac{1}{N}$



How to *efficiently* compute
 $F^{N^{-1}} \cdot V_R$?